

Application Virtualization: An Agent Encapsulation of Software in Virtual Machines to Archive the Execution Performance in Hosts

Yifan Liang

School of Software, Shandong University
Jinan, China P.R.
yifan-liang@outlook.com

Hongjun Dai *

School of Software, Shandong University
Jinan, China P.R.
dahogn@sdu.edu.cn

Abstract—Virtualization is now a key technology for increasing the utilization of physical hardware, and many software programs are benefiting from it. Virtual machine software has become more and more demanding in terms of virtual machine performance. Nowadays, the performance improvement of software inside a virtual machine comes from it of the virtual machine hardware, which is far less than the performance improvement of the same software on the physical machine. We propose Application Virtualization to improve the execution of virtual machine software inside the virtual machine. It can be implemented on existing virtualization platforms to simplify the deployment difficulty and execution efficiency of software inside virtual machines and providing a common way to invoke host software.

Keywords—para-virtualization, application virtualization, inter-domain communication

I. INTRODUCTION

Virtualization is a key technology to increase the granularity and efficiency of physical hardware allocation, and more applications are now executed in Virtual Machines (VMs) or containers. This technology is critical to the development of cloud computing [1], [2], big data [3], [4], [5], and machine learning [6], [7]. This situation has led to significant developments in virtualization technologies, such as lightweight VMs or containers[8] (e.g., katacontainer, docker), lightweight service design, which are used to provide more specific and efficient services.

The key to the success of virtualization technology is the ability to make VMs run on the proper resources and to change them dynamically. This dynamic maximizes resource utilization [9], especially for external devices that cannot be easily changed online (e.g., CPU, memory, GPU, etc.). For the constantly changing load of a VM, the user can add or subtract resources to the VM based on the load of the VM without stopping it. Or modify the memory and CPU of it to reduce the wasted resources when it's idle.

It is not practical to provide a large amount of resources to the VM while it is running. The operating system inside the VM will use the excess memory as cache to improve system performance, and the file system will write the nodes of the file system evenly to the virtual disk making it impossible to simply reduce the redundant space in the future.

Now the hypervisor can adjust the resources of VMs according to the load of them[10]. But this only partially solved the efficiency problem. In [10], each VM run only one service. The real usage of VMs usually needs to run multiple services, and the load of these services are constantly changing. Simply splitting multiple services to different VMs will waste the resources of the host [11], [12]. Therefore, it is necessary to find a new virtualization model: Application Virtualization (AV) to solve the problem of unpredictable load caused by multiple software in a virtualized environment, which makes it impossible to dynamically adjust VM resources. the innovations of this method are:

- Providing a generalized mechanism for VMs to invoke the host software.
- Partially transferring the workload of the VM to the host machine.
- Dynamically expanding the host machine to expand the load capacity of VM without making them offline.

AV is an architecture-independent and focuses on virtualization of software. This virtualization method shows that, in addition to the resource change of the VM itself, the overall efficiency of virtualization can be improved by changing the software execution path inside the VM. Also, transferring some workload inside the VM to the host can reduce resource consumption and the response time of software in it to external requests.

II. BACKGROUND AND RELATED WORK

Virtualization had been widely used in intelligent transportation [13], [14], tele-health care [15], [16], [17], cyber security [18], [19], [20], social services [21], [22], and manufacture [23], [24], [25].

Virtualization technologies, especially full virtualization and para-virtualization technologies, focus on virtualization and management of hardware or kernel components, such as memory[26], [27], [28], GPU[29], file system[30], FPGAs[31]. In addition there are virtualized communication protocols to improve performance[32], [33], [34], [35], [36], [37]. These optimizations are applied to the underlying hardware inside the VM to improve the speed of the software inside it. Qiu's group [38] had proposed a novel variable partitioning

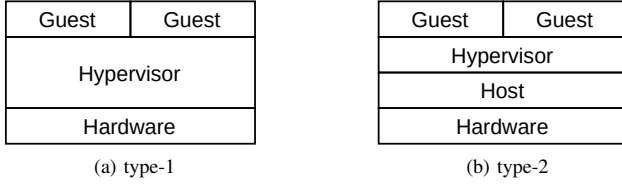


Fig. 1. Two types of hypervisor

and scheduling method for MPSoC (Multi-Processor System on Chip) with virtually shared scratch pad memory in 2013. Then they proposed another high reliable real-time bandwidth scheduling for virtual machines with hidden Markov predicting in tele-health platform in 2015 [39].

A. Overview of Virtualization

Virtualization is a technology used to allocate and share the resources of a physical machine to multiple operating systems to improve the utilization of physical machine resources. The virtual machine manager (VMM, also called hypervisor) does the work required for virtualization, including the emulation of virtual hardware [40], the allocation of physical resources [41], [42], [43], and the translation of run-time instructions. The guest operating system (guest OS) is the operating system that executes the virtual hardware virtualized in the hypervisor.

Hypervisor is the software needed to virtualize the hardware of a physical machine. As shown in the figure 1, there are two forms of hypervisor that exist on physical machines: Type-1 (also known as bare metal) and Type-2 (also known as host-managed) [44]. Type-2 hypervisors exist as a user-state program running on the operating system. This type of VM requires a host operating system (host OS) to help the hypervisor complete the virtualization work.

The essence of virtualization technology is isolation and access control while ensuring resource sharing. The isolation of virtualization consists of the isolation between VMs and the isolation between hypervisor and VMs. Virtualization access control depends on the hypervisor emulation of the hardware. All hardware operations of the VM are translated into hypervisor operations on the hardware, which are performed by the hypervisor or the host operating system.

The main methods used to build VMs are para-virtualization and full virtualization: full virtualization technology virtualizes the entire physical machine, and all devices perceived by the guest OS inside the VM are emulated by the hypervisor. Para-virtualization is similar to full virtualization in that it also virtualizes the entire VM, but in the process of virtualizing the hardware I/O, the I/O instructions are transferred to the host kernel or hypervisor for direct execution [45]. In addition, if the Guest OS can sense that it is in a virtual environment, it can also invoke programs located in the host or other VMs with the help of the hypervisor. In that case not only the hardware on which the VM runs is virtual, but also some of the software it runs is virtual.

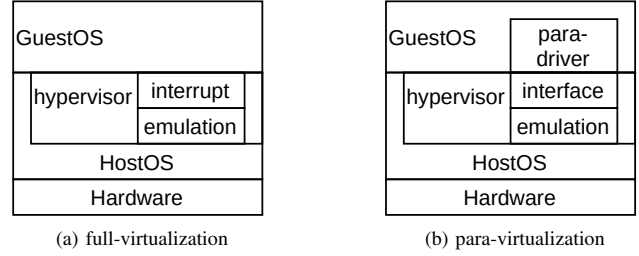


Fig. 2. Virtualization overview

B. Full virtualization

A fully virtualized VM runs in a fully virtualized hardware environment[46]. It usually hosts a full operating system, which is responsible for all operations inside the VM. The guest OS only perceives the virtualized hardware of the hypervisor and runs as if it were running on the physical hardware. Each VM will be completely independent of the hypervisor and will not interfere with each other.

As shown in the figure 2a, in a full virtualization scenario, the hypervisor virtualizes all the hardware in the VM. During the execution of the VM, each request related to the hardware or privileged instruction generates a virtual interrupt. The VM uses virtual interrupts to transfer execution rights to the hypervisor. The hypervisor or the host will return the data to the VM after executing the requests for the hardware. Fully virtualized VMs typically use hardware virtualization instructions to speed up the execution of privileged instructions during run-time. The main examples of full virtualization are KVM (Qemu), VMware, VirtualBox, Hyper-V, and Xen.

The advantage of full virtualization is that the vms are completely independent of the hypervisor and the host, and there is no interference between the VMs. Because the fully virtualized hardware is completely virtualized by hypervisor, a fully virtualized VM can have a different architecture with the physical hardware, without the need to modify the guest OS. The disadvantage is that it is inefficient, with a performance loss of about 33% compared to an operating system running directly on physical hardware without the support of hardware virtualization instructions(such as Intel VT-D and AMD IOMMU)[47].

C. Para-virtualization

Para-virtualization is a similar method to full virtualization, but with optimized I/O paths. Para-virtualized VMs use not only the usual fully virtualized hardware, but also some special virtualized hardware that needs para-virtualization driver. This hardware applies a shorter I/O path compared to the fully virtualized hardware [45], as figure 2b shows. Examples of para-virtualization implementation are Xen[45], Virtio[48].

Compared to full virtualization, the advantage of para-virtualization is that it is more efficient. It's has almost the same I/O performance as native operating system. The disadvantage is that it reduces the isolation between the VMs and the hypervisor. At the same time, para-virtualization requires a

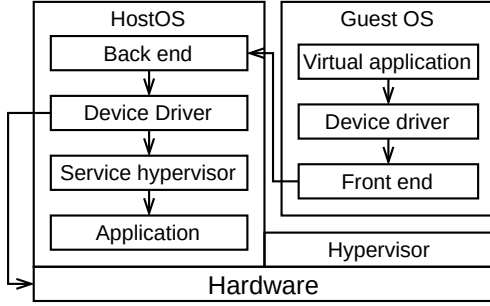


Fig. 3. Architecture Overview of Application Virtualization

specific para-virtualization driver running in the kernel state, which requires the support of the hypervisor and the guest. Para-virtualization, like full virtualization, can only emulated hardware, but not the software.

III. OVERVIEW OF APPLICATION VIRTUALIZATION

A. Overview of the Framework

In general, in the AV framework, only the host has the real programs and data, and all VMs have only the corresponding virtual programs.

As shown in Figure 3, the VM initiates a virtual program call (VPC) through the virtual program, which is then received by the kernel module and forwarded to the host's kernel. The VPC is a program invocation method defined by the AV framework, which can be used to abstract the host program calls. The application virtualization service hypervisor (AVSH) in the host firstly takes the request from the host kernel through the exported device interface and resolves the parameters and VM identifier needed for the host program execution from the request. Then it check the execution permissions of the guest. AVSH will send the execution parameters to the host program for execution and obtains the return result if the guest has the permission to execute the host program. Otherwise, it will prevent the VPC request from executing and generate error report. It is then returned the result to the VM. This whole process completes the execution of a host program inside a VM and can therefore be considered as a virtualization of the software.

B. Deploying Process

The process of using AV is divided into three steps: installing the AV framework, deploying the AVSH plug-in and the corresponding host program, and writing the virtual program.

a) Installing the AV Framework: The only thing needed to do for the VM is to compile and load the AV front-end kernel module. In the host, you need to compile and load the AV back-end compiler module, and you also need to compile and install the management daemon (AVSH) and start it after the back-end module is loaded in the host.

b) Deploying the AVSH plug-in and the Corresponding related program: According to the module interface provided by AVSH, write the plug-in used by AVSH and make it load, and install the host program that the plug-in needs to communicate with and start it before running AVSH.

c) Writing the Virtual Software: The transparency of the framework can be further improved by writing virtual program that relies on the syscall *ioctl* interface exported by the AV framework in the VM. By writing the same interface as the program and passing the execution to the host, the VM is able to use the virtual program with maximum transparency.

C. Advantages

Compared to the other two virtualization methods, AV has three advantages: software virtualization, wide applicability, low resource consumption and requirement, and easy expansion.

a) Software virtualization: This is the biggest advantage AV has. As it can virtualize host software with inter-domain communication, it's possible to use host software in guest with the best performance. This is almost impossible for the other two virtualization methods.

b) Wide applicability: Since AVSH can operate on any accessible entity (e.g., hardware, programs, files, etc.) of the host, AV can virtualize all the accessible entities in the host by writing related AVSH plug-ins. This virtualization capability gives AV a wide range of applicability and allows it to be adapted to various virtualization scenarios.

c) Low resource consumption and requirement: The main workload of the virtual program implemented with the help of AV is undertaken by the host. Since there is no virtualization overhead for the execution of the host application, the virtual application and virtual machine can reduce the consumption of physical resources during their execution. The difficulty of program deployment inside the virtual machine is reduced because hardware requirements for program deployment can be carried by the host.

d) Easy expansion: Due to the fact that the real software in the AV framework is installed in the host OS, the dynamic expand of the VM workload can be achieved by dynamic expansion of the host. So there is no need to modify the VMs in the AV framework. Meanwhile, a full(para)-virtualized VM must add resources if it want more workload.

D. Application

The main application scenario of AV is that multiple VMs share some middleware, such as database middleware (Mysql, Oracle database), application server middleware (IBM WebSphere). The AV export this middleware to the VM to interact with the program inside it. The middleware is deployed in the physical machine to ensure performance, and only the software that uses the middleware and need isolation is inside the VM. The AV emulates the interface of the middleware for programs in the VM, and it passes the VPC to the middleware for processing in the physical machine.

IV. IMPLEMENTATION

The implementation of the AV framework requires the implementation of a Linux guest driver and a Linux host driver. It also need a userspace program in the host. The kernel module model of AV is based on the paravirtualization split driver model, which is divided into a frontend driver in the guest and a backend driver in the host. Besides implementing and installing the kernel driver, the AV framework does not require additional modifications to the host and guest kernels.

A. Implementation in guest OS

As figure 4a and 4b show, AV implements a kernel module in the guest kernel to handle requests and data responses from virtual program in the VM. This module has the ability to construct, merge, forward VPC requests and cache its responses. The module also provides a userspace interface to the user space for VPC invocation.

a) User Interface: The AV exports a misc char device interface to the user space for user to send the VPC request and receive the respond. While initiating a request, the user use the sends the request structure to the kernel and get request identifier allocated by AV through syscall *ioctl*. While querying, the user first get execution status and the return data length through the syscall *ioctl* with the request identifier. The user then allocates enough memory to hold the data according to the length of the data and queries again to extract the VPC respond data.

b) VM Description Structure: The structure consists of the VM identity, mapping-page cache, communication structure, request queue, and VPC respond cache tree. The VM identity is used to identify the VM for the backend; the mapping-page cache is used to cache the page structures used for memory mapping. The communication structure is used to pass messages between the host and the VM, may be implemented with sockets provided by the kernel or para-virtualized message queues. The request queue is used to merge requests for virtual program execution from the user state. The cache tree is used to cache the data received from host OS.

c) Driver Initialization: Driver initialization includes initialization of the communication VM description structure, negotiation with the host, and initialization of the user interface.

The initialization of the VM description structure starts with filling in the VM identity, followed by pre-allocating a small number of pages for the page cache. Then the communication structure will be initialized. The communication structure uses a para-virtualized message queue *virtqueue*, *Xen I/O ring* or, in full virtualization, the communication mechanism provided by the hypervisor to transfer data. The VM then negotiates the communication encrypt key and the maximum number of supported pages to be sent through the communication structure with the host. After the negotiation is complete, the user state device interface is exported and the initialization is completed.

d) Request Construction: The module needs the virtual program identifier and the parameters of invoking the host software from userspace to build the VPC request structure. The kernel module firstly fill the request structure with the virtual program identifier. then it allocates enough memory pages and copies the parameters in the userspace to the allocated pages. These memory pages are mapped to the the host OS kernel later. An index page is also constructed and filled with the mapping identifiers generated during the previous memory mapping. Finally, the structure is sent to the request queue. Once constructed, the structure's identity is return to the userspace.

e) Request Merging: Request Merging is done by the request queue, as the figure 5 shows. The request queue is a FIFO queue, which is implemented by a chain table, and the elements stored in the chain table are the structures constructed during the request construction. The structures are pushed into the queue during the request construction in chronological order. The sender extracts the request one by one to ensure the right execution order.

f) Request Forward: The request queue will notify the sender If it has a request to send. Each time a request is sent, the sender takes the structure at the top of the request queue. The sending method is different depending on the type of VM: para-virtualized VMs use message queues; fully virtualized VMs use the hardware or protocols in the VM that can communicate with the hypervisor.

g) Response Caching and Reading: After the data from the Host is returned, a signal is emitted. This signal wakes up the receiver, which parses the data to find the request identifier, the page identifier and the request execution status. Then it allocates a cache tree node, map the memory and copies the mapped data to this node. After copying, the receiver will cancel the page mapping and release the memory allocated for parsing. Finally, the allocated node is mounted on the cache tree and a signal is emitted to the thread waiting for the data with the same request identifier.

The cache tree is a Binary Search Tree (BST) that uses the request identifier as the query key and contains a waiting queue for user to wait data. In Linux kernel, the red-black tree is chosen since it has a balanced performance while inserting and searching. The guest OS select service identifier as the key of cache tree. The receiver inserts the obtained respond data into the tree as nodes. When querying, the user first queries the tree for its request. If it is not found, it sleeps on the waiting queue. If there is a query node in the tree, the node is removed and the data on it is copied to the userspace. After that, the node is released.

B. Implementation in host OS

AV implements a kernel module and a userspace daemon AVSH in the host OS, as figure 4a and 4c show. The kernel module is responsible for receiving and buffering VPC requests passed from the guest VM and constructing and forwarding replies to them. AVSH focuses on interacting

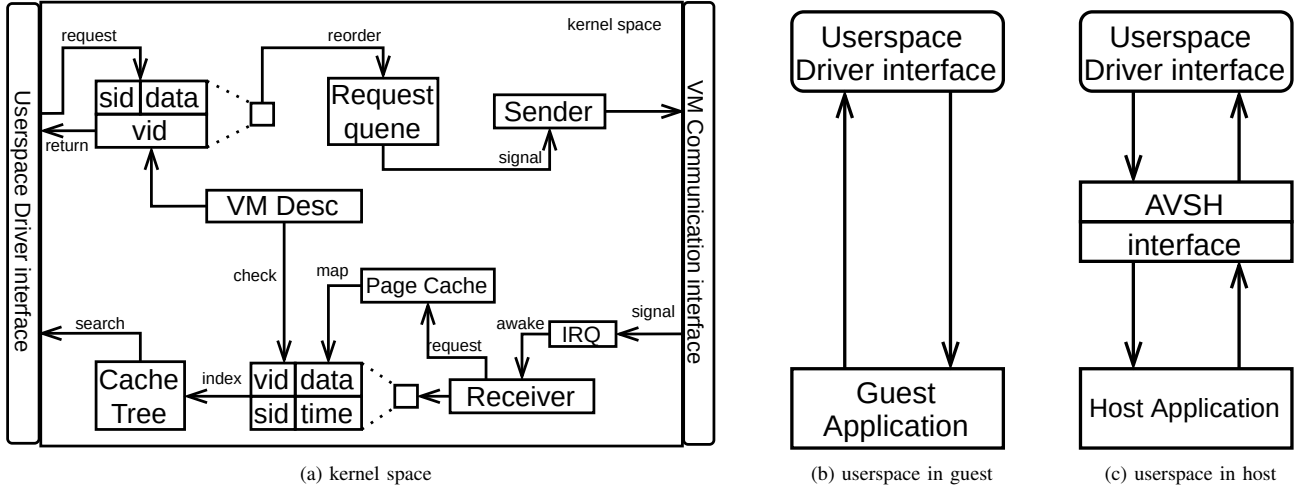


Fig. 4. AV implementation overview

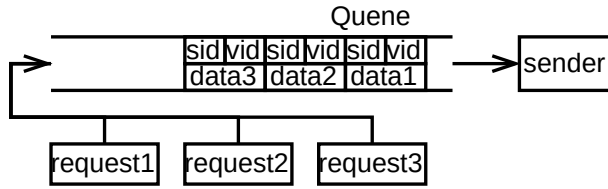


Fig. 5. AV request queue

with the kernel module and sending requests down to the host programs for execution. Since the supplement has many similarities with it of the VM, only the differences between the host and the guest will be focus on.

a) User Interface: Compared to the guest OS, the request is obtained in a different way. In the host OS, the user interface is only available to AVSH. And the request identifier is not specified when querying, but is directly requested from the kernel.

b) VM Description Structure: Host has a description structure for each VM, which forms a chain table. Each description structure contains a communication structure, a request queue and a VM identifier. The cache tree and the mapping page cache table are shared between these description structures.

c) Driver Initialization: Driver initialization in the host is like it in the guest. It also includes initialization of the communication VM description structure and initialization of the user interface. Besides these, the host also need to initialize the cache of VM Description Structure and the related data to negotiation with the guest kernel.

The initialization of the host driver starts with initializing the VM description structure cache. Then the driver will export the misc driver interface to the user space. It initialize the cache tree after the driver interface in export.

d) Guest Negotiation: While in the negotiation with guest, the backend driver first allocates a virtual machine description structure from its cache. Then it generates the symmetric key to be used for the data transfers, and the maximum number of memory pages which can be processed by the backend at a time. This data will send to the guest to decide the optimal arguments to run the framework. After the negotiation, the host driver will start the sender thread and the receiver thread and start processing.

e) AVSH Initialization: AVSH is a system daemon that is started after the loading AV kernel module. At startup, AVSH first loads the modules that interacts with the host programs, reads the permission table. After that, AVSH requests data from the kernel via the syscall *ioctl* and ready to start processing.

f) Request Caching and Reading: The request caching use the same mechanism as the guest module, but the query key of the cache tree is replaced by the union of the request identifier and the VM identifier. The cache tree node stores not only the request identifier and the passed data, but also the VM identifier and the VM description structure.

g) Request Execution: After extracting the request from the kernel, AVSH select the corresponding service module according to the service identifier (sid in the figure). Then it send the VM identifier (vid) and the request data to the host program and wait for the return result. When the result is obtained, the return data and the VM identifier are sent to the kernel to build a reply to the request.

h) Respond Process: The reply is constructed in the same way as the request in the guest module. The responses also are merged by the response queue, as the implementation of the request in the figure 4a. Instead of sending the reply directly, the reply is forwarded to the VM's sender based on the VM description structure in the cache tree. The data is passed to the request queue in the VM description structure and sent to

the VM by the sender thread associated with the VM.

C. Implementation-related key points

In the implementation of AV, there are several key points that have been focused on: efficiency, reliability, and scalability.

a) **Efficiency:** Efficiency is the most important point in the implementation of AV. The optimization of the implementation efficiency includes three aspects: reducing data copies, request indexing, and improving the efficiency of data transfer between VMs and hosts. First, the data of AV requires a double switch from the VM to the client, so at least two data copies are needed to reduce the transfer handle usage between the VM and the host. The data copy between user space and kernel space can be avoided by using memory mapping. Secondly, the cache tree in AV improves the efficiency in finding the data by indexing the service identifiers of it. Finally, AV requires a large amount of data exchange between the guest and the host. In order to reduce the time required for data exchange, we use multiple transfer queues for parallel transfers and a single data copy to release the shared handle before it is processed by the user program.

b) **Reliability:** The reliability of AV is mainly based on the order-preserving execution of VPC. As the execution of VPCs may have execution order dependency, AV needs to preserve execution order. The execution order is preserved by the request queue in the AV kernel module, where all the VPC requests obtained by the AV frontend are sorted in the request queue in chronological order. The sender thread will take VPC requests from the request queue in turn and send them to the backend for execution. This ensures that software with execution order dependencies is executed correctly.

c) **Scalability:** The extension of AV to the host program is implemented with the help of the AVSH service module. This service module defines how to interact with the actual program and the program functions that can be called by the guest programs. When AVSH is started, it will load the module to complete the registration of the program. If you need to use a new host program in the guest OS. The program can directly call the misc device exported by AV kernel module by filling in the new program's identity with system call *ioctl*.

V. EXPERIMENTS AND RESULTS

A. Environment

All experiments were performed on a 16-core Intel Xeon Silver 4123 CPU with 3.00 GHz and 32 GiB of DDR4. We used exactly one VM with 4 VCPUs and 2 GiB memory. Host and Guest are on the same physical disk.

In the hypervisor, we ran Arch Linux (20210115) with the supplied Linux kernel 5.10.7-arch1-1.x86_64 and Xen, compiled with "gcc (GCC) 10.2.1 20201109 (Arch Linux 10.2.0-4)". Inside the VM, we ran Arch Linux (20210115) with a Linux kernel 5.10.7-arch1-1. The kernel config used was based on 5.10.7-arch1-1.x86_64s applied by Arch Linux package linux-headers. We used "gcc (GCC) 10.2.1 20201109

TABLE I
EXECUTION TIME OF EACH PHASE OF AV WHEN EXECUTE SQL

Phase	Execution time(ns)	Percentage
Request: Entering kernel	12069	0.0019%
Request: Preparing to send	10889088	1.7374%
Request: Finishing mapping	10235858	1.6332%
Request: Finishing sending	798811	0.1275%
Request: Extracting to userspace	8109624	1.2939%
Execution: Executing program	591436096	94.3655%
Respond: Entering kernel	3749449	0.5982%
Respond: Preparing to send	823668	0.1314%
Respond: Finishing mapping	151496	0.0242%
Respond: Finishing sending	25371	0.0040%
Respond: Extracting to userspace	518758	0.0828%

(Arch Linux 10.2.0-4)" to compile the Linux kernel modules. The database program for both the host OS and the guest OS was mariadb 10.5.8-1 provided by Arch Linux. SMT and AMD Turbo CORE were enabled. PVHVM was enable in the guest.

B. Experiments and Results

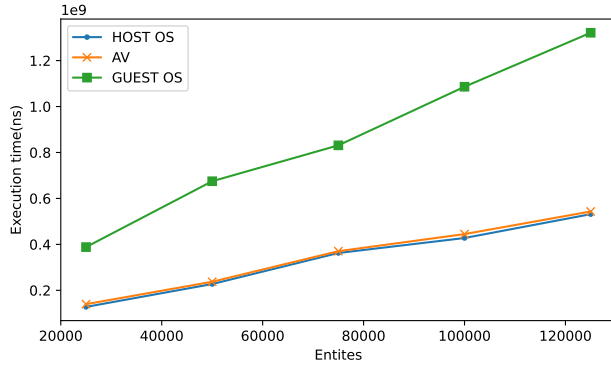
We implemented our method and test the time consumption of each execution stage while executing a sql that inserts 125000 entities into the database. And we also tested the time it takes to execute sql with different number of entities inserted under guest OS, host OS and AV.

The table I shows a special case. In this case, the VM send 3305640 bytes to the host and the host reply only 16 bytes to the VM. Executing the host program uses 94.3655% of time for a total of 591ms. The sum of time the AV framework consumes is 35ms in this case. In detail, the request process takes about 21ms, the respond process take 1ms and 13ms is taken to transfer data between the kernel space and the userspace.

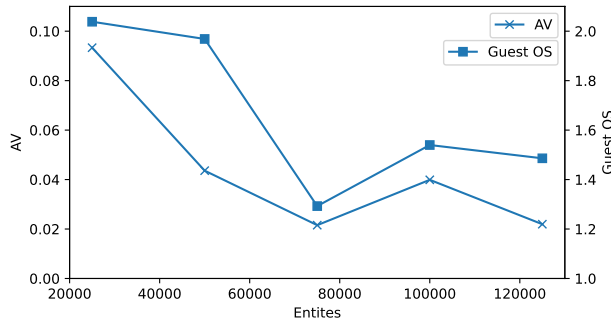
It is obvious that the execution time of the data in the request and reply stage depends on the size of the transmitted data and has a linear relationship in this table. Due to the implementation, the data is split into multiple 4K blocks for easy filling of the page structure and memory alignment. So this result is expected.

Compare the time of extracting data into userspace in request process and respond process, we find it take about 7.5ms to transfer data from kernel space to userspace. It shows that the main time consumption in the syscall is the data copy process, compare to the syscall time consumption in the guest OS. It can also be noticed that the guest OS takes much longer to prepare and transfer data than the host. This is because the guest OS has more data to transfer than the host on the one hand, and because there is also some virtualization cost for the guest to execute instructions. So it is apparent that executing program in the AV framework is much better than in the guest.

The figure 6a shows the time consumption to execute sql with different lengths of data entities inserted in different environments (guest OS, host OS and the AV framework).



(a) Execution time(ns)



(b) Extra execution time percentage

Fig. 6. Execution information about AV, HOST and Guest

It's clearly the execution time in host OS is minimal, follow by the execution in AV framework. And guest OS has the most execution time in all cases, more than two times that of host OS and AV framework. In All cases, the time consumed basically increases linearly with the increase of data size. Besides the execution time in AV is almost the same as in the host, unlike the longer execution time in the virtual machine. The difference between the two is only a few tens of microseconds. It means the AV framework take only a small amount in the VPC execution.

The figure 6b shows the percentage of time consumption to the execution time in the host. It's obviously the percentage of transmission time of AV framework keeps decreasing tendency, from 9.3% to 2.1%, with the increase of data size. This percentage of extra execution time of the guest OS is also has a decreasing tendency, which is from 203.8% to 145.8%. This figure also shows extra execution time of the guest is about 20 times longer than the transmission time of AV. As a result, this figure shows the program is executed more efficiently in AV framework than in the virtual machine.

In general, The execution time of the AV framework is up to 60ms in the worse case, if both the request process and take response process need to transfer upper limit of the data carried by a single request. the execution time of the request is from 1ms to 20ms, depending on the size of data to be sent. And around 45-55% is spent in the guest and the left is almost

all spent in the host. Only a small part of time is consumed by the hypervisor. The biggest variability is contributed by the request sending stage and mapping stage, which consume about 95% of time in total. This is mainly due to these state need to loop all the pages to share or map them, which needs many times of hypercall. This is the same for the response process and execution state switching in kernel and userspace.

In addition to the reduction in execution time, the utilization of CPU in the guest OS also decrease significantly when using AV framework, from nearly 100% to almost 0%. This is because the hosts takes most of computing tasks. As a result, The CPU utilization increased in the host OS. But as the figure 6a shows, it not use so much CPU as the hypervisor.

VI. CONCLUSIONS

In this paper, we first introduced the main ways of building VMs: full virtualization and para-virtualization. We compared the differences between these two types of VMs in terms of I/O mechanisms. We also pointed out that both of them are focused on virtualizing hardware but not software, which leads to the fact that the software inside the VM can only optimize the VM hardware to improve performance. We then proposed AV to solve this software optimization problem inside VMs. By moving the actual software execution inside the host, we can quickly execute the software with the ability of the host to access the hardware directly. We implemented the AV framework with Xen and tested it with database software. And experimental results showed that executing programs in the AV framework is more efficient than executing them directly in the guest kernel.

In future work, the AV framework will use more reliable VM identifiers to identify VMs, instead of directly using the identity assigned by the hypervisor. Because AV needs to dynamically generate exchange keys and other data at runtime, it is not possible to perform hot migration while the VM is running, but in the future, it is possible to try to persist virtual program calls for hot migration. In terms of efficiency, AV now can deliver a maximum of 4MiB of data in a single request, we will try to increase the amount of data that can be requested in a single request to meet the needs of high I/O programs in the future. In terms of adaptability, the AV framework has only been tested on database workloads for now, and we will test more workload models to better adapt to other different applications in the future.

ACKNOWLEDGMENTS

This work has been partially supported by the Natural Science Foundation of Shandong Province (No. ZR2019LZH010) and the project of the Independent Innovation Engineering of Shandong Province (No. 2020CXGC010201).

REFERENCES

- [1] W. Dai, L. Qiu *et al.*, "Cloud infrastructure resource allocation for big data applications," *IEEE Trans. on Big Data*, vol. 4, no. 3, pp. 313–324, 2016.
- [2] M. Qiu, Z. Ming, J. Wang, L. T. Yang, and Y. Xiang, "Enabling cloud computing in emergency management systems," *IEEE Cloud Computing*, vol. 1, no. 4, pp. 60–67, 2014.

- [3] J. Li, M. Qiu, J. Niu, W. Gao, Z. Zong, and X. Qin, "Feedback dynamic algorithms for preemptable job scheduling in cloud systems," in *IEEE/WIC/ACM Int'l Conf. on Web Intelligence (WI'10)*, 2010.
- [4] K. Zhang, J. Kong *et al.*, "Multimedia layout adaptation through grammatical specifications," vol. 10, no. 3, pp. 245–260, 2005.
- [5] R. Lu, X. Jin, S. Zhang *et al.*, "A study on big knowledge and its engineering issues," *IEEE Trans. on Knowledge and Data Eng.*, vol. 31, no. 9, pp. 1630–1644, 2018.
- [6] K. Gai and M. Qiu, "Reinforcement learning-based content-centric services in mobile sensing," *IEEE Network*, vol. 32, no. 4, pp. 34–39, 2018.
- [7] —, "Optimal resource allocation using reinforcement learning for iot content-centric services," *Applied Soft Computing*, vol. 70, pp. 12–21, 2018.
- [8] S. Soltesz, H. Pözl, M. E. Fiuczynski, A. Bavier, and L. Peterson, "Container-based operating system virtualization: A scalable, high-performance alternative to hypervisors," *SIGOPS Oper. Syst. Rev.*, vol. 41, no. 3, p. 275–287, Mar. 2007.
- [9] M. Qiu, Z. Chen, and M. Liu, "Low-power low-latency data allocation for hybrid scratch-pad memory," *IEEE Embedded Systems Letters*, vol. 6, no. 4, pp. 69–72, 2014.
- [10] Y. Guo, A. L. Stolyar, and A. Walid, "Online vm auto-scaling algorithms for application hosting in a cloud," *IEEE Trans. on Cloud Computing*, vol. 8, no. 3, pp. 889–898, 2020.
- [11] T. Wang, Q. Han, S. Sha *et al.*, "On harmonic fixed-priority scheduling of periodic real-time tasks with constrained deadlines," in *IEEE/ACM 53rd Annual Design Automation Conf. (DAC)*, 2016, pp. 1–6.
- [12] M. Qiu, W. Dai, and A. Vasilakos, "Loop parallelism maximization for multimedia data processing in mobile vehicular clouds," *IEEE Trans. on Cloud Computing*, vol. 7, no. 1, pp. 250–258, 2016.
- [13] H. Qiu, Q. Zheng, M. Msahli *et al.*, "Topological graph convolutional network-based urban traffic flow and density prediction," *IEEE Trans. on Intell. Trans. Sys.*, 2020.
- [14] M. Zhu, X. Liu, F. Tang *et al.*, "Public vehicles for future urban transportation," *IEEE Trans. on Intell. Trans. Sys.*, vol. 17, no. 12, pp. 3344–3353, 2016.
- [15] H. Qiu, M. Qiu, M. Liu, and G. Memmi, "Secure health data sharing for medical cyber-physical systems for the healthcare 4.0," *IEEE Jour. of biomedical and health informatics*, vol. 24, no. 9, pp. 2499–2505, 2020.
- [16] Q. Zhang, T. Huang *et al.*, "A case study of sensor data collection and analysis in smart city: provenance in smart food supply chain," *Int'l Journal of Distributed Sensor Networks*, vol. 9, no. 11, p. 382132, 2013.
- [17] M. Qiu, E. Khisamutdinov, Z. Zhao *et al.*, "Rna nanotechnology for computer design and in vivo computation," *Philosophical Transactions of the Royal Society A*, 2013.
- [18] Y. Li, Y. Song, L. Jia *et al.*, "Intelligent fault diagnosis by fusing domain adversarial training and maximum mean discrepancy via ensemble learning," *IEEE Trans. on Indu. Info.*, vol. 17, no. 4, pp. 2831–2842, 2021.
- [19] H. Qiu, T. Dong, T. Zhang *et al.*, "Adversarial attacks against network intrusion detection in iot systems," *IEEE Int. of Things J.*, pp. 1–9, 2021.
- [20] W. Dai, M. Qiu *et al.*, "Who moved my data? privacy protection in smartphones," *IEEE Comm. Magazine*, vol. 55, no. 1, pp. 20–25, 2017.
- [21] M. Qiu, K. Gai, and Z. Xiong, "Privacy-preserving wireless communications using bipartite matching in social big data," *Future Generation Comp. Systems*, vol. 87, pp. 772–781, 2018.
- [22] M. Qiu, D. Cao, H. Su, and K. Gai, "Data transfer minimization for financial derivative pricing using monte carlo simulation with GPU in 5G," *IEEE Int'l Journal of Comm. Systems*, vol. 29, no. 16, pp. 2364–2374, 2016.
- [23] X. Tang, K. Li *et al.*, "A hierarchical reliability-driven scheduling algorithm in grid systems," *Journal of Parallel and Distributed Computing*, vol. 72, no. 4, pp. 525–535, 2012.
- [24] M. Qiu and J. Li, "Real-time embedded systems: optimization, synthesis, and networking," *CRC Press*, 2011.
- [25] K. Lin, W. Wang, Y. Bi *et al.*, "Human localization based on inertial sensors and fingerprints in the industrial internet of things," vol. 101, pp. 113–126, 2016.
- [26] D. Hildenbrand and M. Schulz, "Virtio-mem: Paravirtualized memory hot(un)plug," in *Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ser. VEE 2021, New York, NY, USA, 2021, p. 1–14.
- [27] A. Amiri Sani, K. Boos, S. Qin, and L. Zhong, "I/o paravirtualization at the device file boundary," *SIGPLAN Not.*, vol. 49, no. 4, p. 319–332, Feb. 2014.
- [28] A. Panwar, R. Achermann, A. Basu, A. Bhattacharjee, K. Gopinath, and J. Gandhi, "Fast local page-tables for virtualized numa servers with vmtools," in *26th ACM Int'l Conf. on Arch. Support for Prog. Lang. and Oper. Sys.*, ser. ASPLOS 2021, New York, NY, USA, 2021, p. 194–210.
- [29] M. Yu, C. Zhang, Z. Qi, J. Yao, Y. Wang, and H. Guan, "Vgris: Virtualized gpu resource isolation and scheduling in cloud gaming," in *22nd Int'l Symp. on High-Performance Para. and Dis. Computing*, ser. HPDC '13, New York, NY, USA, 2013, p. 203–214.
- [30] K. Lee, D. Lee, and Y. I. Eom, "A paravirtualized file system for accelerating file i/o," in *2014 Int'l Conf. on Big Data and Smart Computing (BIGCOMP)*, 2014, pp. 309–313.
- [31] W. Wang, M. Bolic, and J. Parri, "pvpfpga: Accessing an fpga-based hardware accelerator in a paravirtualized environment," in *Int'l Conf. CODES+ISSS*, 2013, pp. 1–9.
- [32] Z. He, D. Wang, B. Fu, K. Tan, B. Hua, Z.-L. Zhang, and K. Zheng, "Masq: Rdma for virtual private cloud," in *Conference of the ACM SIG on Data Comm. on the App., Tech., Arch., and Pro. for Computer Comm.*, ser. SIGCOMM '20, New York, NY, USA, 2020, p. 1–14.
- [33] D. Wang, B. Fu, G. Lu, K. Tan, and B. Hua, "Vsocket: Virtual socket interface for rdma in public clouds," in *15th ACM SIGPLAN/SIGOPS Int'l Conf. on Virtual Execution Envi.*, ser. VEE 2019, New York, NY, USA, 2019, p. 179–192.
- [34] A. O. Nordal, r. Kvalnes, R. Pettersen, and D. Johansen, "Streaming as a hypervisor service," in *7th Int'l Workshop on Virt. Tech. in Dist. Computing*, ser. VTDC '13, New York, NY, USA, 2013, p. 33–40.
- [35] A. Nordal, r. Kvalnes, and D. Johansen, "Paravirtualizing tcp," in *6th Int'l Workshop on Virt. Tech. in Dis. Computing Date*, ser. VTDC '12, New York, NY, USA, 2012, p. 3–10.
- [36] Y. Kuperman, E. Moscovici, J. Nider, R. Ladelsky, A. Gordon, and D. Tsafir, "Paravirtual remote i/o," *SIGARCH Comput. Archit. News*, vol. 44, no. 2, p. 49–65, Mar. 2016.
- [37] L. Zeng, Y. Wang, X. Fan, and C. Xu, "Raccoon: A novel network i/o allocation framework for workload-aware vm scheduling in virtual environments," *IEEE Trans. on Par. and Dist. Systems*, vol. 28, no. 9, pp. 2651–2662, 2017.
- [38] L. Zhang, M. Qiu, W. Tseng, and E.-M. Sha, "Variable partitioning and scheduling for MPSoC with virtually shared scratch pad memory," *Journal of Signal Processing Systems*, vol. 58, no. 2, pp. 247–265, 2010.
- [39] J. Wang, M. Qiu, and B. Guo, "High reliable real-time bandwidth scheduling for virtual machines with hidden markov predicting in telehealth platform," *Future Generation Computer Systems*, vol. 49, pp. 68–76, 2015.
- [40] M. Qiu, K. Zhang, and M. Huang, "An empirical study of web interface design on small display devices," in *IEEE/WIC/ACM Int'l Conf. on Web Intelligence (WI'04)*, 2004, pp. 29–35.
- [41] Y. Guo, Q. Zhuge, J. Hu *et al.*, "Optimal data allocation for scratch-pad memory on embedded multi-core systems," in *IEEE International Conference on Parallel Processing (ICPP)*, 2011, pp. 464–471.
- [42] —, "Data placement and duplication for embedded multicore systems with scratch pad memory," *IEEE Trans. on CAD*, 2013.
- [43] J. Niu, C. Liu *et al.*, "Energy efficient task assignment with guaranteed probability satisfying timing constraints for embedded systems," *IEEE Trans. on Parallel and Distributed Systems*, vol. 25, no. 8, pp. 2043–2052, 2013.
- [44] VMware, "What is a hypervisor? — vmware glossary," online; accessed 20-May-2021. [Online]. Available: <https://www.vmware.com/topics/glossary/content/hypervisor>
- [45] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, "Xen and the art of virtualization," *SIGOPS Oper. Syst. Rev.*, vol. 37, no. 5, p. 164–177, Oct. 2003.
- [46] D. Barrett and G. Kipper, "1 - how virtualization happens," in *Virtualization and Forensics*, D. Barrett and G. Kipper, Eds. Boston: Syngress, 2010, pp. 3–24.
- [47] KVM, "Virtio/block/latency — kvm," 2010, online; accessed 7-April-2021. [Online]. Available: <https://www.linux-kvm.org/page/Virtio/Block/Latency>
- [48] R. Russell, "Virtio: Towards a de-facto standard for virtual i/o devices," *SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, p. 95–103, Jul. 2008.